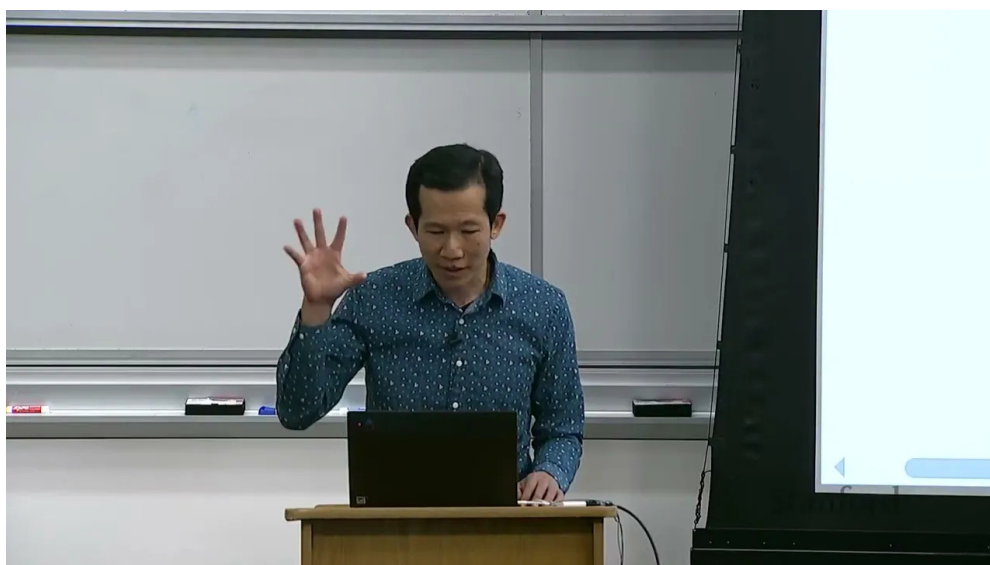


课程笔记

CS336 Lecture 2: Building a Model in PyTorch

基于 Tatsu Hashimoto 授课内容整理

2025 年 4 月



视频作者/频道: Stanford Online

发布日期: 2025-04

视频时长: 1:19:18

项目主页: <https://github.com/hqh1025/ai-course-notes>

目录

1 引言：从概览走向实现	3
1.1 这一讲的阅读方式	3
1.2 一个先算清楚的 napkin math	3
1.2.1 Worked Example: 70B 模型在 1024 张 H100 上训练多久	4
1.2.2 Worked Example: AdamW 训练 7B 模型的内存分解	5
2 Memory Accounting	5
2.1 张量是深度学习的基本单位	5
2.2 Worked example: 一个小张量到底花多少内存	6
2.3 Worked example: 为什么激活值会突然吃掉显存	6
2.4 常见浮点类型	7
2.4.1 Worked Example: 混合精度节省了多少计算和内存	8
2.5 不同模型规模的内存占用对比	8
2.6 GPU 内存层次与带宽	9
2.7 CPU 与 GPU 的内存位置	10
2.8 数据位置的局限	10
2.9 Tensor storage, stride 和 view	10
2.10 View 的机制细节	11
2.11 切片、转置和掩码	11
2.12 逐元素运算和矩阵乘法	11
2.13 内存碎片与 OOM 调试	12
2.14 Einops 和维度命名	13
3 Compute Accounting	13
3.1 FLOPs 和 FLOP/s	13
3.2 线性模型的 FLOPs	13
3.3 更一般的近似	14
3.4 不同 GPU 代际的算力对比	14
3.5 模型 FLOPs 利用率 MFU	14
3.6 Worked example: 把 FLOPs 直接换成训练时间	15
3.7 反向传播的 FLOPs	15
3.8 Worked example: 两层线性网络的训练步	16
4 Models	16
4.1 <code>nn.Parameter</code>	16
4.2 初始化：为什么要按输入维度缩放	16
4.3 自定义模块	17
4.4 Transformer 参数量估算	17
4.5 模型参数、缓存和 checkpoint 的关系	18

5	Training Loop and Best Practices	18
5.1	随机性与可复现	18
5.2	数据加载：把序列切成 batch	19
5.3	Pinned memory 和异步拷贝	19
5.4	优化器	19
5.5	Adam 的更新规则	20
5.6	Optimizer 选择的实际含义	20
5.7	训练时的内存核算	21
5.8	训练循环	21
5.9	梯度累积：用小 batch 模拟大 batch	22
5.10	学习率调度	23
5.11	梯度裁剪	23
5.12	训练循环的状态机	24
5.13	训练监控与日志	24
5.14	Checkpointing	26
5.14.1	Checkpoint 频率与存储成本	26
5.14.2	Checkpoint 的异步保存	27
5.14.3	Checkpoint 格式选择	27
5.14.4	长训练的 Checkpoint 最佳实践	28
5.15	Checkpoint 调试清单	28
5.16	混合精度训练	29
5.16.1	混合精度的核心思想	29
5.16.2	PyTorch AMP 实现	29
5.16.3	混合精度下的完整内存分析	30
5.16.4	FP8 训练：下一代低精度前沿	30
5.17	Activation Checkpointing（梯度检查点）	31
5.18	混合精度 + Activation Checkpointing 的协同	32
6	总结与延伸	33
6.1	本讲知识体系	33
6.2	核心公式速查	33
6.3	从本讲到后续课程的衔接	34
6.4	拓展阅读	34

1 引言：从概览走向实现

上一讲先概览了语言模型、tokenization 以及从零构建模型的动机。本讲进入真正的实现层面，目标不是再讲 Transformer 的整体结构，而是把训练模型所需要的 **PyTorch 原语** 一层层搭起来。

本讲的三件事

1. 从张量开始，搞清楚模型参数、梯度、优化器状态和数据到底如何占用内存。
2. 从矩阵乘法开始，学会对 FLOPs 和 FLOP/s 做资源核算。
3. 从 ‘nn.Parameter’、‘nn.Module’、优化器和训练循环出发，搭出一个最小但完整的训练系统。

讲义里的核心方法很朴素：**先算账，再写代码**。如果不知道一个张量有多少字节、一次矩阵乘法有多少 FLOPs、一次训练步会额外产生多少梯度和优化器状态，就很难判断模型为什么慢，也很难知道什么时候该换数据类型、换实现方式或者换训练策略。

1.1 这一讲的阅读方式

这一讲最值得反复看的不是某一段代码，而是背后的判断方式：

- **定义**：这个对象是什么，PyTorch 里对应什么类型。
- **直觉**：为什么它会占内存，为什么它会花算力。
- **机制**：PyTorch / CUDA / autograd 实际怎么做。
- **应用**：这个原语在训练系统里被怎么使用。
- **局限**：这个近似或实现在哪里会失效。

阅读顺序建议：**Definition** → **Intuition** → **Mechanism** → **Application** → **Limitations**。

两类资源

本讲只盯住两种资源：

- **内存**：参数、梯度、激活值、优化器状态都要装得下。
- **计算**：训练和推理的成本通常都能归结成 FLOPs。

1.2 一个先算清楚的 napkin math

先看两个典型估算。

1. 训练一个 70B 参数模型，在 15T tokens 上，用 1024 张 H100，大概需要多久？
2. 如果只看显存，8 张 80GB H100 上，使用朴素 AdamW，最多能训练多大的 dense 模型？

6ND 规则——训练 FLOPs 的万能近似

对于密集 Transformer，训练的总 FLOPs 可以用一个极其简洁的公式近似：

$$C \approx 6 \times N \times D$$

其中 N 是模型参数量， D 是训练 token 数。这里的 6 来自前向传播 ($\approx 2ND$) 加反向传播 ($\approx 4ND$)。

这个公式忽略了 embedding 层、attention mask 计算、layernorm 等非矩阵乘法操作，但在规模估算时误差通常不超过 10%–20%。它是做训练预算时最重要的单个公式。

1.2.1 Worked Example: 70B 模型在 1024 张 H100 上训练多久

我们把上面的估算展开为一步一步的计算：

Step 1: 计算总 FLOPs

模型参数量 $N = 70 \times 10^9$ ，训练数据量 $D = 15 \times 10^{12}$ tokens。由 6ND 公式：

$$C = 6 \times 70 \times 10^9 \times 15 \times 10^{12} = 6.3 \times 10^{24} \text{ FLOPs}$$

Step 2: 计算集群有效算力

H100 SXM 在 bfloat16 下的峰值算力约为 $989.5 \times 10^{12} \text{ FLOP/s} \approx 10^{15} \text{ FLOP/s}$ 。假设 MFU = 0.5（即有效利用率 50%），则单卡有效算力为：

$$\text{单卡有效} = 10^{15} \times 0.5 = 5 \times 10^{14} \text{ FLOP/s}$$

1024 张卡的集群每天提供的 FLOPs：

$$\text{日 FLOPs} = 1024 \times 5 \times 10^{14} \times 86400 = 4.42 \times 10^{22} \text{ FLOPs/day}$$

Step 3: 换算训练天数

$$\text{训练天数} = \frac{6.3 \times 10^{24}}{4.42 \times 10^{22}} \approx 143 \text{ 天}$$

参数	数值	来源
模型参数量 N	70×10^9	模型架构定义
训练 token 数 D	15×10^{12}	训练数据量
总 FLOPs $C = 6ND$	6.3×10^{24}	6ND 公式
H100 bf16 峰值	$\sim 10^{15} \text{ FLOP/s}$	NVIDIA 规格表
假设 MFU	0.5	经验估计
GPU 数量	1024	集群规模
训练天数	$\sim 143 \text{ 天}$	总 FLOPs / 日 FLOPs

表 1: 70B 模型训练时间估算的完整参数表

MFU 的敏感性

上面的计算中 MFU 设为 0.5，但实际 MFU 可能在 0.3–0.6 之间波动。如果 MFU 从 0.5 降到 0.35，训练时间就会从 143 天变成 204 天——差出两个月。因此 MFU 的微小变化会导致训练成本的巨大波动，这也是工程优化的核心目标之一。

1.2.2 Worked Example: AdamW 训练 7B 模型的内存分解

以一个 7B 参数模型为例，详细分解使用 AdamW 优化器时的 GPU 显存占用。

假设：使用混合精度训练，参数和梯度为 bf16 (2 bytes)，优化器状态中的主权重 (master weights) 和一阶/二阶矩为 fp32 (4 bytes)。

$$N = 7 \times 10^9 \text{ 个参数}$$

组件	dtype	每参数字节	7B 模型内存
模型参数 (训练副本)	bf16	2	14 GB
梯度	bf16	2	14 GB
主权重 (master weights)	fp32	4	28 GB
一阶矩 m_t	fp32	4	28 GB
二阶矩 v_t	fp32	4	28 GB
静态总计		16	112 GB

表 2: AdamW 混合精度训练 7B 模型的内存分解 (不含激活值)

这意味着仅模型参数和优化器状态就需要 112 GB 显存，还没有算激活值。以 80 GB 的 H100 来算，**至少需要 2 张卡**才能装下静态状态，实际上考虑激活值可能需要 4 张或更多。

如果用纯 fp32 训练 (不做混合精度)，每个参数需要：

$$4 (\text{参数}) + 4 (\text{梯度}) + 4 (m_t) + 4 (v_t) = 16 \text{ bytes}$$

7B 模型静态内存为 $7 \times 10^9 \times 16 = 112 \text{ GB}$ ，和混合精度的总量恰好相同——但混合精度的优势在于前向/反向计算速度更快、激活值更小。

为什么混合精度还要存 fp32 主权重

混合精度训练的核心思想是：用低精度做前向和反向计算以加速，但用 fp32 精度累积梯度更新。如果直接在 bf16 参数上做梯度更新，微小的更新 (如 $\eta \cdot g \ll 1$) 会因为 bf16 的有限精度被截断为零，导致训练停滞。fp32 主权重保证了累积更新的精度。

2 Memory Accounting

2.1 张量是深度学习的基本单位

在 PyTorch 里，几乎所有东西最终都是张量：

- 模型参数
- 梯度
- 优化器状态
- 数据
- 激活值

张量的两件事

一个 tensor 的内存成本只看两件事：

- 元素个数 ‘`numel()`’
- 每个元素占多少字节 `element_size()`

因此最简单的内存估计就是

$$\text{memory} = \text{numel} \times \text{element size}.$$

2.2 Worked example: 一个小张量到底花多少内存

先看一个最小例子。一个 4×8 的 float32 tensor 有 32 个元素，每个元素 4 字节，所以总内存是：

$$4 \times 8 \times 4 = 128 \text{ bytes}.$$

这类算式很小，但它的逻辑和大模型完全一样。训练系统里的很多 bug，本质上就是“把 128 bytes 的直觉误搬到 128 GB 的规模上”。

张量	形状	dtype	内存
Toy matrix	4×8	float32	128 B
Hidden state batch	$32 \times 2048 \times 4096$	bfloat16	$\approx 512 \text{ MB}$
FFN weight	4096×16384	float32	$\approx 256 \text{ MB}$
Adam states	same as weight	float32 $\times 2$	$\approx 512 \text{ MB}$

表 3: 从玩具张量到大模型参数，内存公式本质上没有变

实战里最常见的误解

很多人只看参数量，不看激活值和 optimizer state。实际上，训练时最贵的往往不是模型前向本身，而是“参数 + 梯度 + 优化器状态 + 激活值”四者叠加后的总账。

2.3 Worked example: 为什么激活值会突然吃掉显存

参数内存是静态的，激活值内存是随 batch size 和 sequence length 线性增长的。对于一个形状为 $B \times L \times H$ 的激活张量，如果使用 bfloat16，它的内存大约是：

$$2 \times B \times L \times H \text{ bytes}.$$

这就是为什么你明明觉得“模型参数没那么大”，训练时还是会 OOM：真正爆掉的可能是中间激活，而不是权重本身。

对象	典型形状	每元素字节数	增长方式
参数	固定形状	2 或 4	随模型规模增长
激活值	$B \times L \times H$	2 或 4	随 batch 和序列长度增长
梯度	与参数同形状	2 或 4	每次反向都要存
优化器状态	与参数同形状	4 或更多	只和参数量相关

表 4: 训练显存最重要的差别: 参数是固定成本, 激活值是动态成本

为什么这件事和 Assignment 1 直接相关

你在实现 Transformer 时, batch size、sequence length、hidden size、attention head 数都会联动影响激活值的总量。只盯参数量, 往往会把最关键的显存瓶颈看漏。

2.4 常见浮点类型

浮点数的表示由三部分组成: **符号位**(sign)、**指数位**(exponent) 和 **尾数位**(mantissa/significand)。不同数据类型在这三部分的分配上做了不同的权衡。

类型	字节	指数位	尾数位	动态范围	训练常见用途
float32 (FP32)	4	8	23	$\sim 10^{\pm 38}$	主权重、优化器状态
float16 (FP16)	2	5	10	$\sim 10^{\pm 5}$	部分前向/激活
bfloat16 (BF16)	2	8	7	$\sim 10^{\pm 38}$	训练和推理的主流选择
fp8 (E4M3)	1	4	3	$\sim 10^{\pm 2}$	前沿训练/推理实验
fp8 (E5M2)	1	5	2	$\sim 10^{\pm 5}$	梯度计算

表 5: 常见数值类型的位分配与动态范围对比

这里最重要的结论是: **bfloat16 不是”更小的 float32”, 而是”保留了 float32 的指数范围, 但牺牲一些尾数精度”**。在深度学习里, 这种取舍通常比 float16 更稳。

bf16 vs fp16: 看起来都是 2 字节, 但行为完全不同

fp16 只有 5 位指数, 动态范围仅约 $10^{\pm 5}$, 这意味着绝对值小于 $\sim 6 \times 10^{-8}$ 的数会被截断为零 (underflow), 大于 65504 的数会溢出为 inf。在训练中, 梯度值经常非常小, fp16 的 underflow 会导致梯度”消失”, 必须配合 **loss scaling** 来人为放大 loss, 使梯度落入 fp16 的可表示范围。

bf16 有 8 位指数, 动态范围和 fp32 一样大 ($\sim 10^{\pm 38}$), 因此**不需要 loss scaling**。代价是尾数只有 7 位 (fp16 有 10 位), 精度较低。但实践表明, 对于大多数深度学习任务, 动态范围比尾数精度更重要。

结论: 如果硬件支持 bf16 (如 A100、H100), 优先选 bf16 而非 fp16。

为什么训练比推理更难低精度化

训练时会同时涉及参数、梯度、优化器状态和反向传播链式法则。数值误差会在更新过程中累积，因此训练对数值稳定性更敏感；推理阶段没有反向传播，通常可以更激进地量化。

2.4.1 Worked Example: 混合精度节省了多少计算和内存

考虑一个矩阵乘法 $Y = XW$ ，其中 $X \in \mathbb{R}^{4096 \times 4096}$ ， $W \in \mathbb{R}^{4096 \times 4096}$ 。

内存对比：

dtype	X 内存	$X + W$ 总内存
float32	$4096^2 \times 4 = 64 \text{ MB}$	128 MB
bfloat16	$4096^2 \times 2 = 32 \text{ MB}$	64 MB
fp8	$4096^2 \times 1 = 16 \text{ MB}$	32 MB

表 6: 同一矩阵在不同精度下的内存占用

计算吞吐对比（以 H100 SXM 为例）：

dtype	峰值 TFLOP/s	相对 FP32 加速
FP32	67	1×
TF32 (Tensor Core)	495	~ 7×
BF16/FP16 (Tensor Core)	990	~ 15×
FP8 (Tensor Core)	1979	~ 30×

表 7: H100 SXM 在不同数据类型下的理论峰值算力

这解释了为什么混合精度训练不只是“省内存”——它同时带来了巨大的**计算加速**。从 fp32 切到 bf16，理论上可以获得约 15 倍的矩阵乘法加速。

2.5 不同模型规模的内存占用对比

下表展示了从 1B 到 70B 不同规模模型在使用 AdamW 混合精度训练时的静态内存需求（不含激活值）：

模型规模	参数 (bf16)	梯度 (bf16)	主权重 (fp32)	Adam 状态 (fp32)	总计
1B	2 GB	2 GB	4 GB	8 GB	16 GB
7B	14 GB	14 GB	28 GB	56 GB	112 GB
13B	26 GB	26 GB	52 GB	104 GB	208 GB
30B	60 GB	60 GB	120 GB	240 GB	480 GB
70B	140 GB	140 GB	280 GB	560 GB	1120 GB

表 8: 不同规模模型使用 AdamW 混合精度训练的静态内存需求

快速心算规则

对于 AdamW 混合精度训练，每个参数的静态内存约为 $2 + 2 + 4 + 4 + 4 = 16$ bytes。因此：

- 1B 模型 ≈ 16 GB (1 张 H100 刚好)
- 7B 模型 ≈ 112 GB (至少 2 张 H100)
- 70B 模型 ≈ 1120 GB (至少 14 张 H100, 纯装参数和状态)

这还没有算激活值！实际训练通常需要更多显存或使用模型并行。

2.6 GPU 内存层次与带宽

理解 GPU 的内存层次对于优化训练性能至关重要。GPU 的内存系统是分层的，越靠近计算单元速度越快、容量越小：

层级	容量	带宽	说明
寄存器	KB 级	—	每个线程私有
共享内存/L1	128–228 KB/SM	~ 100 TB/s	SM 内共享，用户可编程
L2 缓存	50–60 MB	~ 12 TB/s	全 GPU 共享
HBM (显存)	80 GB	3.35 TB/s	H100 SXM 规格
NVLink	—	900 GB/s	GPU 间互联
PCIe 5.0	—	128 GB/s	CPU–GPU 互联

表 9: H100 SXM GPU 的内存层次 (数值为近似值)

Arithmetic Intensity 与带宽瓶颈

一个操作是**计算瓶颈** (compute-bound) 还是**带宽瓶颈** (memory-bound)，取决于它的 **arithmetic intensity** (每字节数据传输执行的 FLOPs 数)：

$$\text{Arithmetic Intensity} = \frac{\text{FLOPs}}{\text{Bytes accessed}}$$

- **矩阵乘法**：arithmetic intensity 高 ($O(n)$ FLOPs per byte)，通常是 compute-bound，能充分利用 Tensor Core。
- **逐元素操作** (如 ReLU、LayerNorm、softmax)：arithmetic intensity 低 ($O(1)$ FLOPs per byte)，通常是 memory-bound，瓶颈在显存带宽。

这就解释了为什么 Transformer 的矩阵乘法可以通过低精度获得近线性加速，而 LayerNorm 等操作的加速效果有限。

Roofline 模型

Roofline 模型是分析 GPU 性能的经典工具。它给出了给定 arithmetic intensity 下的理论性能上限：

$$\text{Attainable FLOP/s} = \min(\text{Peak FLOP/s}, \text{Bandwidth} \times \text{Arithmetic Intensity})$$

当 arithmetic intensity 低于某个拐点 ($= \text{Peak FLOP/s} / \text{Bandwidth}$) 时，性能受带宽限制；超过拐点后，性能受计算能力限制。对 H100 SXM bf16 而言，这个拐点约为 $990 \text{ TFLOP/s} / 3.35 \text{ TB/s} \approx 295 \text{ FLOPs/Byte}$ 。

2.7 CPU 与 GPU 的内存位置

默认情况下，PyTorch tensor 存在 CPU 内存中。要利用 GPU 的并行计算能力，必须显式把 tensor 迁移到 GPU 显存中。

位置	特点	常见操作
CPU paged memory	默认、可分页、适合常规处理	数据预处理、批生成
CPU pinned memory	页锁定，可异步拷贝到 GPU	数据加载器预取
GPU memory	计算速度最快、容量最宝贵	参数、激活、临时计算结果

表 10: 训练中常见的内存位置

Pinned memory 的意义

把 CPU 上的 batch 放到 pinned memory 后，可以用 `non_blocking=True` 异步拷贝到 GPU。这样可以把“CPU 取下一批数据”和“GPU 处理当前批数据”重叠起来。

2.8 数据位置的局限

CPU 内存、pinned memory 和 GPU 显存各有分工，但也各有上限。CPU 便宜但慢，GPU 快但贵，pinned memory 便于搬运但不是无限大。训练系统经常死在“所有东西都放 GPU”这种简单化思路。真正的工程做法是让数据尽量在正确的位置停留尽量短的时间。

2.9 Tensor storage, stride 和 view

PyTorch tensor 不只是“值的集合”，它更像是：

- 一块连续或者非连续的底层存储；
- 加上一组元数据，告诉你每个维度怎么映射到存储地址；
- 以及形状、dtype、device 等信息。

在二维张量里，‘stride(0)’表示跨行要跳多少元素，‘stride(1)’表示跨列要跳多少元素。对于一个标准行主序矩阵来说，行 stride 通常等于列数，列 stride 通常为 1。

view 是免费的，copy 不是

‘view’、切片和转置很多时候都只是不同的“视图”，不会复制底层数据，因此几乎不耗额外内存。真正触发 ‘contiguous()’ 或显式拷贝时，才会同时增加内存和计算成本。

一个容易踩坑的点是：不是所有视图都能再 ‘view’ 成任意形状。若张量是非连续的，直接 ‘view’ 常常会报错，这时要先调用 ‘contiguous()’。

2.10 View 的机制细节

‘view’ 的前提是底层存储布局能被新的形状解释。如果张量是转置后的非连续布局，就不能直接靠 ‘view’ 改形状。这就是为什么 ‘contiguous()’ 不是多余操作，而是把“逻辑视图”重新落回“物理连续存储”的一步。

为什么这一点重要

很多模型代码看起来只是“换了个形状”，实际上却偷偷复制了一大块张量。只要复制发生得不该发生，内存就会涨，性能也会掉。

2.11 切片、转置和掩码

切片、列选择、转置等操作通常只返回新的视图而不是复制数据。对于 causal attention 这类场景，‘triu’ 很常见，因为它可以快速生成上三角矩阵，用来屏蔽未来 token 的信息。

causal mask 的直觉

如果 $M[i, j] = 1$ 表示位置 i 对位置 j 有贡献，那么自回归语言模型要求未来位置不能泄露到过去，于是只保留上三角或下三角的合法连接。

操作	是否复制数据	常见用途
切片 ‘x[:, 1]’	通常不复制	选列、选 token、选 channel
转置 ‘transpose’	不复制，但可能非连续	改变矩阵方向
‘contiguous()’	会复制	把非连续视图落回连续内存
‘triu()’	生成新张量	causal mask、上三角约束

表 11: 哪些操作免费、哪些会复制，是内存优化的基础判断

2.12 逐元素运算和矩阵乘法

逐元素运算包括加法、乘法、平方、开方、‘triu’ 等。这类操作一般是线性的，FLOPs 规模约为 $O(mn)$ 。

真正的深度学习主角还是矩阵乘法：

$$X \in \mathbb{R}^{B \times D}, \quad W \in \mathbb{R}^{D \times K}, \quad Y = XW \in \mathbb{R}^{B \times K}.$$

这里的 FLOPs 约为

$$2BDK,$$

因为每个输出元素都需要一次乘法和一次加法。

运算	复杂度	备注
逐元素加/乘	$O(mn)$	规模线性
矩阵乘法	$O(mnp)$	通常是计算瓶颈
反向传播	约为前向的 2 倍到 3 倍	依赖具体图结构

表 12: 训练里最常见的计算量级

当输入张量多出 batch 维和 sequence 维时, PyTorch 的矩阵乘法会自动在前面的维度上广播并批处理。也就是说, 只要最后两维满足矩阵乘法规则, 前面的维度会被视作并行的“外层循环”。

2.13 内存碎片与 OOM 调试

GPU 内存并不是一个可以任意使用的大池子。PyTorch 的 CUDA 内存分配器会缓存已释放的内存块以加速后续分配, 但这也会导致**内存碎片化**——总可用内存看起来足够, 但没有足够大的连续块来满足一次大的分配请求。

OOM 类型	表现	排查方法
真实 OOM	显存确实不够	减小 batch size 或用模型并行
碎片化 OOM	总量够但连续块不够	<code>torch.cuda.memory_summary()</code>
泄漏式 OOM	显存随步数缓慢增长	检查是否把 tensor 挂在 history 上
激活值爆炸	某步突然 OOM	检查输入序列长度异常值

表 13: 常见的 GPU OOM 场景与排查方法

常见的内存泄漏陷阱

以下写法会意外保留计算图, 导致内存持续增长:

- 把 loss 直接 append 到 Python list 中 (应使用 `loss.item()`)
- 在循环外部持有中间 tensor 的引用而不 detach
- 忘记调用 `optimizer.zero_grad(set_to_none=True)`, 导致梯度在多步间累积

PyTorch 提供了一套实用的内存调试工具:

```
1 # 查看当前显存使用
2 print(torch.cuda.memory_allocated() / 1e9, "GB_allocated")
3 print(torch.cuda.memory_reserved() / 1e9, "GB_reserved")
4
5 # 打印详细内存摘要
6 print(torch.cuda.memory_summary())
7
8 # 记录内存快照用于分析
9 torch.cuda.memory._record_memory_history()
```

```

10 # ... run some code ...
11 torch.cuda.memory._dump_snapshot("mem_snapshot.pickle")

```

Listing 1: GPU 内存调试常用命令

2.14 Einops 和维度命名

原生 PyTorch 容易在 ‘-2’、‘-1’ 这种负索引里把维度写乱。‘einops’ 的价值在于把维度变成可读的名字。

维度命名的价值

- ‘batch seq hidden’ 一眼就知道张量语义。
- ‘einsum’ 可以把广播和求和写得更清楚。
- ‘reduce’ 和 ‘rearrange’ 让 reshape 逻辑更不容易出错。

这讲里最常见的三种写法是：

- ‘einsum(x, y, "batch seq1 hidden, batch seq2 hidden -> batch seq1 seq2")’
- ‘reduce(x, "... hidden -> ...", "sum")’
- ‘rearrange(x, "... (heads hidden1) -> ... heads hidden1", heads=2)’

3 Compute Accounting

3.1 FLOPs 和 FLOP/s

FLOP 是一次浮点操作，比如一次加法或乘法。FLOP/s 则是硬件每秒能执行多少浮点操作。这里要避免两个缩写混淆：

- **FLOPs**：算了多少浮点操作。
- **FLOP/s**：每秒能做多少浮点操作。

资源核算的核心问题

训练时间约等于

$$\frac{\text{总 FLOPs}}{\text{有效 FLOP/s}}$$

所以除了算法本身，我们还必须知道硬件的有效利用率。

3.2 线性模型的 FLOPs

先从最简单的线性模型开始。若有 B 个样本、每个样本维度为 D 、输出维度为 K ，那么前向传播的矩阵乘法

$$Y = XW$$

大约需要

$$2BDK$$

个 FLOPs。

后向传播则需要对参数和中间激活分别做链式法则，整体大约是前向的两倍，所以线性层的完整训练步常用近似是：

$$\text{forward} + \text{backward} \approx 6BDK.$$

为什么是 6

粗略地看：

- 前向：一次乘法 + 一次加法，约 $2BDK$
- 反向：对参数和输入各再做一次类似的乘加，约 $4BDK$

合起来就是约 $6BDK$ 。

3.3 更一般的近似

对于 Transformer 这类模型，虽然结构更复杂，但一阶近似下，训练 FLOPs 仍可以写成：

$$6 \times (\text{token 数}) \times (\text{参数量}).$$

这也是前面 napkin math 的来源。

3.4 不同 GPU 代际的算力对比

选择硬件时，不仅要看峰值 FLOP/s，还要看内存容量和带宽。不同代际 GPU 的关键参数对比如下：

GPU	HBM	带宽	BF16 Peak	FP8 Peak	年份
V100 SXM	32 GB	900 GB/s	—	—	2017
A100 SXM	80 GB	2.0 TB/s	312 TF	—	2020
H100 SXM	80 GB	3.35 TB/s	990 TF	1979 TF	2023
H200 SXM	141 GB	4.8 TB/s	990 TF	1979 TF	2024
B200 SXM	192 GB	8.0 TB/s	2250 TF	4500 TF	2025

表 14: NVIDIA 数据中心 GPU 关键参数对比 (TF = TFLOP/s)

为什么 H200 和 H100 算力相同但价值更高

H200 和 H100 使用相同的 GPU 芯片 (GH200)，BF16 算力完全相同。H200 的优势在于将 HBM 从 80 GB 升级到 141 GB、带宽从 3.35 TB/s 升级到 4.8 TB/s。对于大模型推理 (memory-bound 场景)，更大的显存意味着可以加载更大的模型，更高的带宽意味着更快的 token 生成速度。

3.5 模型 FLOPs 利用率 MFU

硬件规格表给的是“峰值”性能，但真实训练不可能一直跑满。于是引入：

$$\text{MFU} = \frac{\text{actual FLOP/s}}{\text{promised FLOP/s}}.$$

MFU 不是一个抽象学术指标，而是直接决定你有多少硬件预算被真正转化成训练速度。一般来说，MFU 大于 0.5 已经算不错，尤其当矩阵乘法占主导时更是如此。

为什么 float32 和 bfloat16 的 FLOP/s 不一样

硬件规格里的“峰值 FLOP/s”强依赖数据类型。很多 GPU 对 bfloat16/float16 的吞吐量远高于 float32，所以同一块卡上，切换 dtype 会显著改变你的实际吞吐。

3.6 Worked example: 把 FLOPs 直接换成训练时间

如果你知道总 FLOPs 和硬件的有效 FLOP/s，就可以直接估计 wall-clock time。以 70B 参数、15T tokens 的训练为例：

$$6 \times 70 \times 10^9 \times 15 \times 10^{12}$$

是总 FLOPs。若用 1024 张 H100，且把有效利用率设成 0.5，那么每天能提供的 FLOPs 就是：

$$1024 \times 0.5 \times \text{H100 peak FLOP/s} \times 86400.$$

把前者除以后者，就得到大约 144 天。

量	公式	含义
总 FLOPs	$6NT$	N 是参数量， T 是 token 数
日 FLOPs	$G \cdot \eta \cdot 86400$	G 是峰值 FLOP/s， η 是 MFU
训练天数	总 FLOPs/日 FLOPs	直接换算 wall-clock

表 15: 把算力预算换成时间，是训练计划里最实用的一个公式

这个估算的局限

这个公式没有把通信、数据等待、checkpoint 开销、非矩阵乘法算子和 pipeline bubble 细节全部展开，所以它是预算级估算，不是秒级预测。

3.7 反向传播的 FLOPs

以两层线性模型为例：

$$x \xrightarrow{w_1} h_1 \xrightarrow{w_2} h_2 \rightarrow \text{loss}.$$

前向已经需要两次矩阵乘法。反向传播时，对 w_2 、 h_1 、 w_1 以及输入的梯度分别做链式法则，整体 FLOPs 约为前向的两倍。因此整个训练步常见的经验法则仍然是：

$$\text{train step FLOPs} \approx 6 \times (\text{tokens}) \times (\text{parameters}).$$

这个近似什么时候够用

只要你在做的是规模估算、预算估算、训练时长估算，这个近似就足够好。它不是为了证明定理，而是为了让你能快速判断“这个实验值不值得跑”。

3.8 Worked example: 两层线性网络的训练步

假设输入 $x \in \mathbb{R}^{B \times D}$ ，第一层权重 $w_1 \in \mathbb{R}^{D \times D}$ ，第二层权重 $w_2 \in \mathbb{R}^{D \times K}$ 。那么：

- 前向： xw_1 再乘 w_2
- 反向：对 w_2 、 h_1 、 w_1 求梯度

如果你把每个矩阵乘法都按 2 FLOPs / 输出元素来算，最后就会得到“前向约 2 倍参数乘 token 数、反向约再来 4 倍”的总量级。这个例子说明：复杂模型的 FLOPs 近似，本质上是许多小矩阵乘法的叠加。

阶段	主要操作	FLOPs 级别	含义
前向	两次 matmul	$2BD(D + K)$	算预测
反向	对参数和输入求导	约为前向的 2 倍	传播梯度
训练步	前向 + 反向	约 $6BD(D + K)$	完整更新一次

表 16: 两层线性网络是理解训练 FLOPs 的最小实例

4 Models

4.1 nn.Parameter

PyTorch 里模型参数通常以 ‘nn.Parameter’ 的形式保存。它和普通 tensor 类似，但会自动被 ‘nn.Module’ 注册为可训练参数，并出现在 ‘parameters()’ 和 ‘state_dict()’ 里。

参数与普通 tensor 的差异

‘nn.Parameter’ 的本质还是 tensor，但它明确表达了“这是要被训练的东西”。这会影响：

- 自动求导
- 模型参数遍历
- checkpoint 保存/恢复

4.2 初始化：为什么要按输入维度缩放

如果直接用高斯随机初始化，输出的尺度会随着输入维度增长而变大。一个简单的做法是除以 $\sqrt{\text{input_dim}}$ ，这样可以让输出的方差大致稳定。

Xavier 的直觉

初始化要尽量让前向传播和反向传播中的数值尺度都保持稳定。最简单的经验就是：参数规模要随输入维度做归一化。

在更实际的实现里，通常会使用 truncated normal，把极端离群值裁掉，进一步减少训练不稳定的风险。

4.3 自定义模块

下面这类简单线性模块很关键，因为它展示了 ‘nn.Module’ 的最小骨架。

```
1 class Linear(nn.Module):
2     def __init__(self, input_dim: int, output_dim: int):
3         super().__init__()
4         self.weight = nn.Parameter(
5             torch.randn(input_dim, output_dim) / np.sqrt(input_dim)
6         )
7
8     def forward(self, x: torch.Tensor) -> torch.Tensor:
9         return x @ self.weight
```

再往上堆叠多个层，就能得到一个简单的深度线性模型。重点不是”线性模型很强”，而是”理解模块化结构如何把参数组织起来”。

4.4 Transformer 参数量估算

理解模型参数的具体构成，对于内存预算和训练规划至关重要。以一个标准的 decoder-only Transformer 为例：

组件	参数量	说明
Token embedding	$V \times d$	V : 词表大小, d : 隐藏维度
每层 QKV 投影	$3 \times d \times d$	三个矩阵各 $d \times d$
每层输出投影	$d \times d$	Attention 输出映射
每层 FFN	$2 \times d \times 4d$	上投影 + 下投影
每层 LayerNorm	$2 \times 2d$	两个 LN 各有 γ, β
LM Head	通常与 embedding 共享	不额外计数
每层合计	$\approx 12d^2$	忽略 LN 的 $O(d)$ 项
L 层总计	$\approx 12Ld^2 + Vd$	

表 17: Decoder-only Transformer 的参数量分解

Worked Example: LLaMA-2 7B 的参数量验证

LLaMA-2 7B 的超参数为 $d = 4096$, $L = 32$, $V = 32000$, FFN 隐藏维度为 11008 (使用 SwiGLU, 实际为 $\frac{8}{3}d$ 上取整)。

- Embedding: $32000 \times 4096 = 131M$
- 每层 Attention: $4 \times 4096^2 = 67M$ (QKV + O)
- 每层 FFN (SwiGLU): $3 \times 4096 \times 11008 = 135M$ (gate + up + down)
- 每层 LN: $2 \times 2 \times 4096 = 16K$ (可忽略)
- 32 层合计: $32 \times (67M + 135M) = 6,464M$
- 最终 LN + embedding: $\approx 131M + 16K$

- 总计: $\approx 6,738M \approx 6.7B$

这和“7B”的命名基本吻合（考虑到 RoPE、bias 等细节的差异）。

12d² 速算法

对于标准 Transformer（FFN 倍率为 4），每层参数约为 $12d^2$ 。因此给定隐藏维度 d 和层数 L ，总参数量（不含 embedding）约为 $12Ld^2$ 。这个公式在做 napkin math 时非常方便：

- $d = 4096, L = 32: 12 \times 32 \times 4096^2 \approx 6.4B$
- $d = 5120, L = 40: 12 \times 40 \times 5120^2 \approx 12.6B$
- $d = 8192, L = 80: 12 \times 80 \times 8192^2 \approx 64.4B$

state_dict 里有什么

`state_dict()` 负责保存模型的状态。对一个模块来说，它通常包括：

- 所有 ‘nn.Parameter’
- 有状态层的缓存

所以 checkpoint 不只是保存参数本身，而是保存整个训练状态。

4.5 模型参数、缓存和 checkpoint 的关系

`state_dict()` 的价值不只是“方便保存文件”。它把训练时需要恢复的所有重要状态显式列出来，避免你把“模型”误以为只是权重。对有些层来说，除了权重还有 running statistics、动量缓存或其他持久状态。若恢复时漏掉这些内容，模型行为就会和上次训练完全不是同一个轨迹。

对象	存什么	恢复时影响什么
Model state	参数和模块缓存	前向输出、收敛路径
Optimizer state	动量、平方梯度等	更新方向、学习率适配
Random seed	RNG 状态	数据顺序、dropout、初始化

表 18: checkpoint 不只是参数文件，而是一份训练状态快照

5 Training Loop and Best Practices

5.1 随机性与可复现

训练里随机性来自很多地方：

- 参数初始化
- dropout
- 数据顺序

因此调试时最好把三套随机种子一次性设好：

- `torch.manual_seed`
- `numpy.random.seed`
- `random.seed`

为什么要统一随机种子

当实验结果不稳定时，最怕的是“同一个 bug 在不同 run 里表现不同”。先锁定随机性，才能把问题缩小到实现本身。

5.2 数据加载：把序列切成 batch

语言模型的数据本质上是一串整数 token。最常见的做法是把它序列化到 numpy 数组，再用‘memmap’懒加载。

```
1 start_indices = torch.randint(len(data) - sequence_length, (batch_size,))
2 x = torch.tensor([data[start:start + sequence_length] for start in start_indices])
```

这段逻辑的语义很直接：

- 随机采样若干起点；
- 从每个起点切出一段固定长度的 token 序列；
- 堆成一个 batch。

为什么要用 memmap

真实数据集可能非常大，不能一次性全读进内存。‘np.memmap’允许只把当前访问到的部分映射到内存中，因此更适合大规模语言建模。

5.3 Pinned memory 和异步拷贝

若 batch 先放在 pinned memory，再执行

```
x.to(device, non_blocking=True),
```

CPU 到 GPU 的拷贝可以异步进行。这样就能把数据搬运和 GPU 计算重叠起来，减少空转。

数据管线也是性能瓶颈

很多时候模型不慢，是因为数据喂不够快。只盯着算子优化而不看数据加载，常常会把训练吞吐的真正瓶颈漏掉。

5.4 优化器

本讲先自己实现了两个最简单的优化器：

- SGD
- AdaGrad

然后再把它们和更常见的优化器对应起来：

- Momentum = SGD + 指数滑动平均
- AdaGrad = SGD + 累积平方梯度
- RMSProp = AdaGrad + 指数滑动平均的平方梯度
- Adam = RMSProp + Momentum

优化器的代价

优化器不仅改变更新规则，还会显式增加状态量。比如 Adam 至少要保存两份额外状态，这就是为什么优化器内存常常比你想象得更贵。

5.5 Adam 的更新规则

Adam 的核心是同时维护梯度的一阶矩估计（均值）和二阶矩估计（未中心化方差），并用偏差校正来补偿初始化时的偏差：

$$\begin{aligned}m_t &= \beta_1 m_{t-1} + (1 - \beta_1) g_t && \text{（一阶矩/动量）} \\v_t &= \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 && \text{（二阶矩/自适应学习率）} \\ \hat{m}_t &= \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} && \text{（偏差校正）} \\ \theta_{t+1} &= \theta_t - \eta \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}\end{aligned}$$

其中 $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$ 是常见默认值。

AdamW vs Adam + L2 正则化——一个微妙但重要的区别

经典 Adam + L2 正则化把权重衰减加在梯度上：

$$g'_t = g_t + \lambda \theta_t, \quad \theta_{t+1} = \theta_t - \eta \cdot \frac{\hat{m}_t(g'_t)}{\sqrt{\hat{v}_t(g'_t)} + \epsilon}$$

问题是：权重衰减项 $\lambda \theta_t$ 也被 Adam 的自适应学习率缩放。对于梯度较大的参数，衰减效果被削弱；对于梯度较小的参数，衰减效果被放大。这并不是我们想要的行为。

AdamW (Loshchilov & Hutter, 2019) 把权重衰减解耦出来：

$$\theta_{t+1} = (1 - \eta \lambda) \theta_t - \eta \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

这样权重衰减不受自适应学习率的影响，对所有参数施加均匀的正则化。

实践结论：现代 LLM 训练几乎都使用 AdamW 而非 Adam + L2。PyTorch 中 `torch.optim.AdamW` 实现的就是解耦版本。

5.6 Optimizer 选择的实际含义

在很小的模型上，SGD、AdaGrad、RMSProp 和 Adam 的差异可能看起来只是收敛速度不同；但在大模型上，它们还直接决定内存占用。一个 optimizer 的”更智能”，通常意味着更多状态、更复杂的更新逻辑，外加更高的调参成本。

优化器	额外状态	每参数内存	超参数	适用场景
SGD	无	0 bytes	η	基线、教学
SGD+Mom.	m	4 bytes (fp32)	η, β	计算机视觉
AdaGrad	$\sum g^2$	4 bytes	η, ϵ	稀疏特征
RMSProp	v	4 bytes	η, β_2, ϵ	非平稳目标
Adam	m, v	8 bytes	$\eta, \beta_1, \beta_2, \epsilon$	通用
AdamW	m, v	8 bytes	$\eta, \beta_1, \beta_2, \epsilon, \lambda$	LLM 训练标配

表 19: 优化器对比：状态数量直接决定内存成本

AdaGrad 的致命缺陷

AdaGrad 累积的是梯度平方和 $\sum_{i=1}^t g_i^2$ ，这个值只增不减。随着训练进行，自适应学习率 $\eta / \sqrt{\sum g^2 + \epsilon}$ 会单调下降趋近于零，最终导致训练停滞。RMSProp 和 Adam 通过指数滑动平均（而非累积求和）解决了这个问题。

5.7 训练时的内存核算

对一个普通的前馈模型，训练时要同时考虑：

部分	数量级	说明
参数	N_p	模型权重
激活值	N_a	前向过程中保存的中间结果
梯度	N_p	每个参数对应一个梯度
优化器状态	N_p 或更多	取决于算法

表 20: 训练步的常见内存组成

若用 float32 朴素训练，则总内存可以粗略写成：

$$4 \times (N_p + N_a + N_p + N_p) = 4(3N_p + N_a).$$

这也是为什么“模型参数看起来装得下”，但“训练时还是爆显存”。

5.8 训练循环

标准训练循环可以压缩成四步：

1. 前向：算预测值和 loss
2. 反向：‘loss.backward()’
3. 更新：‘optimizer.step()’
4. 清零：optimizer.zero_grad(set_to_none=True)

```

1 for t in range(num_train_steps):
2     x, y = get_batch(B=B)
3     pred_y = model(x)
4     loss = F.mse_loss(pred_y, y)
5     loss.backward()
6     optimizer.step()
7     optimizer.zero_grad(set_to_none=True)

```

训练循环的本质

训练循环不是”写起来麻烦”，而是”把前向、反向、更新三种状态转换显式地串起来”。一旦逻辑清楚了，调参和定位 bug 都会容易很多。

5.9 梯度累积：用小 batch 模拟大 batch

当 GPU 显存不足以容纳理想的 batch size 时，**梯度累积** (gradient accumulation) 是最简单的解决方案。其原理是：连续做多次前向 + 反向但不执行 `optimizer.step()`，让梯度在多个 micro-batch 上累积，最后一次性更新参数。

```

1 accumulation_steps = 4 # effective batch = micro_batch * 4
2
3 for step in range(num_train_steps):
4     for micro_step in range(accumulation_steps):
5         x, y = get_batch(B=micro_batch_size)
6         pred = model(x)
7         loss = loss_fn(pred, y) / accumulation_steps # scale loss
8         loss.backward() # gradients accumulate
9
10    optimizer.step()
11    optimizer.zero_grad(set_to_none=True)

```

Listing 2: 梯度累积的标准实现

梯度累积的两个常见错误

- **忘记缩放 loss**：如果不除以 `accumulation_steps`，累积的梯度会比真正大 batch 的梯度大 k 倍 (k 为累积步数)，等效于学习率放大了 k 倍。
- **与 BatchNorm 的冲突**：BatchNorm 的统计量是在每个 micro-batch 上独立计算的，和真正的大 batch 行为不同。LLM 通常使用 LayerNorm，不受此影响。

方法	Micro BS	累积步数	有效 BS	显存占用
直接大 batch	64	1	64	极高
梯度累积	16	4	64	中等
梯度累积	4	16	64	较低
梯度累积	1	64	64	最低

表 21: 梯度累积在数学上等价于大 batch，但显存和吞吐有差异

5.10 学习率调度

现代 LLM 训练几乎都使用学习率调度器 (learning rate scheduler)，而不是固定学习率。最常见的策略是 **warmup + cosine decay**：

1. **Warmup 阶段**：从接近 0 的学习率线性增长到峰值学习率，通常持续总步数的 1%-5%。
2. **Cosine decay 阶段**：学习率按余弦函数从峰值衰减到一个很小的最终值（通常为峰值的 1/10 或更小）。

$$\eta_t = \begin{cases} \eta_{\max} \cdot \frac{t}{T_{\text{warmup}}} & \text{if } t < T_{\text{warmup}} \\ \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min}) \left(1 + \cos \left(\frac{t - T_{\text{warmup}}}{T_{\text{total}} - T_{\text{warmup}}} \cdot \pi \right) \right) & \text{otherwise} \end{cases}$$

为什么需要 warmup

训练初期，Adam 的二阶矩 v_t 尚未充分预热 (bias correction 后的估计仍不稳定)，此时使用大学习率容易导致梯度爆炸或训练发散。Warmup 让模型在参数空间中先”小步试探”，等优化器状态稳定后再加大步长。

经验上，warmup 步数通常设为总步数的 1%-2%，或者 2000 步左右。

5.11 梯度裁剪

梯度裁剪 (gradient clipping) 是防止训练不稳定的重要工具。最常见的做法是**全局梯度范数裁剪**：

$$\hat{g} = \begin{cases} g & \text{if } \|g\| \leq C \\ C \cdot \frac{g}{\|g\|} & \text{if } \|g\| > C \end{cases}$$

其中 C 是裁剪阈值（通常设为 1.0）， $\|g\|$ 是所有参数梯度拼接后的 L2 范数。

```

1 loss.backward()
2 # Clip gradients before optimizer step
3 grad_norm = torch.nn.utils.clip_grad_norm_(
4     model.parameters(), max_norm=1.0
5 )
6 optimizer.step()
7 optimizer.zero_grad(set_to_none=True)
8

```



```

9 # Monitor grad_norm for training stability
10 if grad_norm > 10.0:
11     print(f" Warning: large gradient norm {grad_norm:.2f}" )

```

Listing 3: 梯度裁剪的标准用法

梯度范数是训练健康的晴雨表

监控梯度范数可以帮助诊断训练问题：

- 梯度范数持续增大 → 可能训练发散，需要降低学习率
- 梯度范数突然飙升 → 可能遇到异常数据或数值不稳定
- 梯度范数趋近于零 → 可能学习率太小或梯度消失
- 梯度范数频繁被裁剪 → 裁剪阈值可能设得太低

5.12 训练循环的状态机

训练步可以看成是一个状态机的四个阶段：**Batch ready** → **Forward** → **Backward** → **Optimizer step**，然后循环。真正出问题的地方通常不是其中某一步本身，而是状态切换时忘了清空梯度、忘了保存 optimizer state，或者把数据放在了错误的 device 上。

状态	输入	输出	常见 bug
Batch ready	tokenizer / data loader	batch tensor	放错 device、shape 对不上
Forward	batch + parameters	prediction / loss	dtype 不匹配、mask 错误
Backward	loss	gradients	忘记 requires_grad、梯度爆炸
Step	gradients + optimizer state	更新后的 parameters	忘记 zero_grad、checkpoint 丢 optimizer

表 22: 训练循环里每个状态都对应一类典型 bug

5.13 训练监控与日志

大规模训练中，监控指标是及早发现问题的唯一手段。一个完整的训练监控系统通常需要记录以下信息：

指标	正常行为	异常信号
Training loss	单调下降（整体趋势）	突然跳升、停滞不下、NaN
Learning rate	按调度器变化	始终为 0 或异常大
梯度范数	稳定在某个范围内	持续增大、突然飙升
GPU 利用率	接近 100%	大幅波动或持续低于 50%
显存占用	稳定不变	随步数缓慢增长（泄漏）
吞吐量 (tokens/s)	稳定	周期性下降（可能是 checkpoint I/O)

表 23: 训练监控的核心指标与异常诊断

```

1  import wandb # or tensorboard
2
3  wandb.init(project="cs336-lecture02")
4
5  for step in range(num_train_steps):
6      x, y = get_batch(B=B)
7      pred = model(x)
8      loss = loss_fn(pred, y)
9      loss.backward()
10
11     grad_norm = torch.nn.utils.clip_grad_norm_(
12         model.parameters(), max_norm=1.0
13     )
14
15     optimizer.step()
16     optimizer.zero_grad(set_to_none=True)
17
18     if step % log_interval == 0:
19         wandb.log({
20             "loss": loss.item(),
21             "grad_norm": grad_norm.item(),
22             "lr": optimizer.param_groups[0]['lr'],
23             "gpu_mem_gb": torch.cuda.memory_allocated() / 1e9,
24         }, step=step)

```

Listing 4: 训练循环中的日志记录示例

Loss spike 的常见原因

训练过程中偶尔出现的 loss spike（突然跳升）可能由以下原因引起：

- **异常数据**：某个 batch 包含异常长的序列或罕见 token 组合
- **学习率过大**：尤其在 warmup 结束附近
- **数值不稳定**：梯度中出现 inf 或 nan
- **数据重复**：训练数据中存在大量重复导致过拟合后跳出

轻微的 loss spike 通常可以自行恢复，但如果 loss 持续不下降或出现 NaN，通常需要从上一个 checkpoint 回滚并降低学习率。

5.14 Checkpointing

训练语言模型往往很久，而且很容易中断（硬件故障、抢占、OOM）。所以要定期保存 checkpoint。

```
1 checkpoint = {  
2     'model_state_dict': model.state_dict(),  
3     'optimizer_state_dict': optimizer.state_dict(),  
4     'step': step,  
5     'loss': loss.item(),  
6     'rng_state': torch.random.get_rng_state(),  
7 }  
8 torch.save(checkpoint, f'checkpoint_step_{step}.pt')
```

Listing 5: 保存 checkpoint 的标准写法

```
1 checkpoint = torch.load(f'checkpoint_step_{step}.pt')  
2 model.load_state_dict(checkpoint[{}'model_state_dict'{}])  
3 optimizer.load_state_dict(checkpoint[{}'optimizer_state_dict'{}])  
4 torch.random.set_rng_state(checkpoint[{}'rng_state'{}])  
5 start_step = checkpoint[{}'step'{}]
```

Listing 6: 恢复 checkpoint 的标准写法

只存模型不够

如果只存 model，不存 optimizer，那么恢复后学习率状态、动量累积、平方梯度缓存都没了，训练行为会和之前不一样。尤其是 Adam 系列优化器， m_t 和 v_t 需要大量 step 才能预热到稳定状态，丢失后相当于重新开始预热。

5.14.1 Checkpoint 频率与存储成本

Checkpoint 的频率需要在”恢复代价”和”存储成本”之间权衡：

策略	频率	权衡
保守	每 1000 步	存储开销小，但中断后最多浪费 1000 步
激进	每 100 步	恢复损失小，但存储快速增长
混合	近期密集 + 远期稀疏	保留最近 5 个 + 每 N 步一个永久

表 24: checkpoint 频率策略的常见选择

以 7B 模型为例，一个完整 checkpoint（模型 + 优化器 + RNG）大约需要：

$$\underbrace{7 \times 10^9 \times 2}_{\text{bf16 参数}} + \underbrace{7 \times 10^9 \times (4 + 4 + 4)}_{\text{fp32 主权重} + m + v} \approx 14 + 84 = 98 \text{ GB}$$

如果每 100 步存一个、保留最近 10 个，光 checkpoint 就需要约 1 TB 存储。

5.14.2 Checkpoint 的异步保存

大规模训练中，同步保存 checkpoint 会阻塞训练。常见做法是把 checkpoint 数据先拷贝到 CPU 内存（或 pinned memory），然后在后台线程中异步写入磁盘，同时 GPU 继续下一步训练。

分布式 checkpoint

当模型被切分到多张 GPU 上（模型并行/数据并行）时，每张卡只保存自己的那部分状态。恢复时需要确保切分方式完全一致，否则状态会对不上。PyTorch 的 `DistributedStateDictSaveLoad` 和 DeepSpeed 的 checkpoint 引擎都提供了自动化的分布式 checkpoint 管理。

5.14.3 Checkpoint 格式选择

不同的 checkpoint 格式有不同的权衡：

格式	优点	缺点	适用场景
<code>torch.save</code> (pickle)	PyTorch 原生、支持任意对象	安全风险 (pickle 反序列化)、不支持部分加载	训练中间 checkpoint
<code>safetensors</code>	安全、支持 mmap 部分加载、跨框架	只能存 tensor 字典	模型发布与共享
<code>torch.distributed.save</code> <code>.checkpoint</code>	支持分布式保存/加载、resharding	API 较新、生态不够成熟	大规模分布式训练

表 25: 常见 checkpoint 格式的对比

pickle 的安全风险

`torch.save` 使用 Python 的 pickle 协议，这意味着加载一个不受信任的 checkpoint 文件可能执行任意代码。在加载来自互联网的模型权重时，优先选择 `safetensors` 格式，或者使用 `torch.load(..., weights_only=True)` 来限制反序列化行为。

5.14.4 长训练的 Checkpoint 最佳实践

对于持续数周甚至数月的大规模训练，checkpoint 策略需要更加系统化：

1. **滚动保留**：保留最近 k 个 checkpoint（如最近 5 个），自动删除更早的，防止存储爆炸。
2. **里程碑永久保存**：在关键节点（如每 10% 训练进度）永久保存一个 checkpoint，用于后续分析。
3. **保存训练指标**：除了模型和优化器状态，还要保存 loss 曲线、学习率、梯度范数等指标，方便恢复后验证一致性。
4. **预留时间**：在集群抢占或定时任务到期前，提前触发一次 checkpoint（如收到 SIGTERM 信号时）。
5. **校验完整性**：保存后立即验证 checkpoint 文件的完整性（如 checksum），避免写入中断导致的损坏。

Checkpoint 与训练的 wall-clock 开销

以 7B 模型为例，一个完整 checkpoint 约 98 GB。在 NVMe SSD（写入 3 GB/s）上，同步保存需要约 33 秒。如果每 100 步存一次，且每步约 2 秒，那么 checkpoint 开销约为 $33/(100 \times 2) \approx 16\%$ 。使用异步保存可以将这个开销降到接近零。

5.15 Checkpoint 调试清单

Checkpoint 往往不是“能不能存”这么简单，而是“恢复后是不是同一个实验”。如果恢复后的 loss 曲线明显不一致，通常优先查下面几项：

现象	优先检查	常见原因
恢复后 loss 跳变	optimizer state、学习率调度	只存了 model 没存 optimizer
训练变慢	device / dtype / data loader	batch 没放到 GPU 或 pinned memory
梯度不更新	requires_grad / zero_grad	参数被冻住或梯度没清
结果不可复现	RNG、数据顺序、dropout	随机种子没统一

表 26: checkpoint 的问题通常要从训练状态而不是单纯参数开始排查

5.16 混合精度训练

5.16.1 混合精度的核心思想

混合精度训练 (Mixed Precision Training) 的核心思想是：用低精度做计算密集的部分（前向、反向的矩阵乘法），用高精度做数值敏感的部分（参数更新、loss 计算）。

一个典型的混合精度训练流程：

1. 维护一份 **fp32 主权重** (master weights)
2. 每步开始时，将主权重转换为 bf16 副本
3. 用 bf16 做前向和反向传播（矩阵乘法在 Tensor Core 上加速）
4. 得到 bf16 梯度后，转换回 fp32
5. 在 fp32 精度下更新主权重

混合精度的三重好处

1. **计算加速**：bf16 矩阵乘法在 Tensor Core 上比 fp32 快约 $8\times-16\times$
2. **内存节省**：激活值用 bf16 存储，内存减半
3. **带宽节省**：bf16 数据传输量减半，减少 memory bandwidth 瓶颈

5.16.2 PyTorch AMP 实现

PyTorch 的 Automatic Mixed Precision (AMP) 通过 `torch.cuda.amp` 提供了自动化的混合精度支持：

```
1 scaler = torch.amp.GradScaler() # only needed for fp16
2
3 for x, y in dataloader:
4     with torch.autocast(device_type='cuda', dtype=torch.bfloat16):
5         pred = model(x)
6         loss = loss_fn(pred, y)
7
8     # bf16 does not need scaler, but fp16 does
9     loss.backward()
10    optimizer.step()
11    optimizer.zero_grad(set_to_none=True)
```

Listing 7: 使用 PyTorch AMP 的训练循环

`autocast` 上下文管理器会自动决定哪些操作用低精度、哪些保留 fp32。一般来说：

- 用 **bf16/fp16**：矩阵乘法、卷积、线性层
- 保留 **fp32**：softmax、layernorm、loss 计算、累加操作

Loss Scaling: fp16 的必要补丁

使用 fp16 时，由于动态范围只有 $10^{\pm 5}$ ，小梯度容易 underflow 为零。**Loss scaling** 通过在反向传播前将 loss 乘以一个大数（如 2^{16} ），使梯度值被放大到 fp16 的可表示范围内，然后在参数更新前除回来。

GradScaler 还会动态调整 scale 因子：如果发现梯度中有 inf/nan (overflow)，就减小 scale 并跳过该步更新。

使用 bf16 时不需要 loss scaling，因为 bf16 的动态范围和 fp32 一样大。这是 bf16 在实践中更受欢迎的另一个原因。

5.16.3 混合精度下的完整内存分析

以 N 参数模型为例，对比纯 fp32 训练和 bf16 混合精度训练的内存：

组件	纯 FP32	混合精度	说明
模型参数	$4N$	$2N$ (bf16)	前向用低精度
梯度	$4N$	$2N$ (bf16)	反向产生低精度梯度
主权重	—	$4N$ (fp32)	混合精度额外持有
Adam m_t	$4N$	$4N$	始终 fp32
Adam v_t	$4N$	$4N$	始终 fp32
静态总计	$16N$	$16N$	静态内存相当
激活值	$4 \cdot A$	$2 \cdot A$	激活减半是关键

表 27: 混合精度的主要内存收益来自激活值减半和计算加速

关键洞察：混合精度在静态内存（参数 + 优化器）上并没有显著节省，因为需要额外维护 fp32 主权重。**真正的收益在于：**

1. 激活值内存减半（对于大 batch size 和长序列影响巨大）
2. Tensor Core 的计算加速（ $8\times-16\times$ ）
3. 内存带宽减半

5.16.4 FP8 训练：下一代低精度前沿

FP8 是 H100 及更新 GPU 支持的 8-bit 浮点格式。它有两种变体，各自针对不同的使用场景：

格式	指数位	尾数位	动态范围	设计目的
E4M3	4	3	$\sim 10^{\pm 2}$	前向权重和激活（需要更高精度）
E5M2	5	2	$\sim 10^{\pm 5}$	反向梯度（需要更大范围）

表 28: 两种 FP8 变体的设计权衡

FP8 训练的关键挑战是动态范围非常有限，因此需要 **per-tensor scaling**：为每个 tensor 维护一个缩放因子，使其值域映射到 FP8 的可表示范围内。这增加了实现复杂度，但在 H100 上可以获得相比 BF16 约 2× 的额外加速。

FP8 训练的成熟度

截至 2025 年，FP8 训练仍处于快速发展期。主要挑战包括：

- 缩放因子的选择策略（延迟缩放 vs 即时缩放）
- 某些层（如 attention softmax、LayerNorm）仍需更高精度
- 不同框架（NVIDIA TransformerEngine、MS-AMP）的实现差异
- 训练稳定性在超大规模模型上的验证仍不充分

FP8 在推理中的应用已相当成熟，但在训练中仍需谨慎评估。

5.17 Activation Checkpointing（梯度检查点）

Activation Checkpointing 的核心思想

反向传播需要前向过程中保存的激活值来计算梯度。对于深层网络，所有层的激活值同时驻留在 GPU 内存中，内存开销巨大。

Activation checkpointing（也叫 gradient checkpointing 或 rematerialization）的策略是：只保存部分层的激活值（称为 checkpoint），丢弃其余层的激活值。反向传播到某一层时，从最近的 checkpoint 重新做前向计算来恢复所需的激活值。这是一个经典的时间换空间权衡。

假设模型有 L 层，每层激活值占 a 内存：

策略	激活值内存	额外计算	适用场景
不做 checkpointing	$L \cdot a$	0	内存充足
每 $\sqrt{L}$ 层 checkpoint	$\sqrt{L} \cdot a$	~ 33% 前向	经典权衡
每层 checkpoint	a (仅 1 层)	~ 100% 前向	极端内存受限

表 29: Activation checkpointing 的不同策略

PyTorch 提供了 `torch.utils.checkpoint.checkpoint` 函数来实现这一功能：

```

1 from torch.utils.checkpoint import checkpoint
2
3 class TransformerBlock(nn.Module):
4     def forward(self, x):
5         # ... transformer block logic ...
6         return output
7
8 class Model(nn.Module):
9     def __init__(self, num_layers):

```



```

10     super().__init__()
11     self.layers = nn.ModuleList(
12         [TransformerBlock() for _ in range(num_layers)]
13     )
14
15     def forward(self, x):
16         for layer in self.layers:
17             # Wrap each layer with checkpointing
18             x = checkpoint(layer, x, use_reentrant=False)
19         return x

```

Listing 8: 使用 activation checkpointing 的示例

Activation Checkpointing 的陷阱

- **计算开销**: 每个被 checkpoint 的段在反向时会重新执行前向计算, 总计算量增加约 33% (如果每 $\sqrt{L}$ 层 checkpoint)。
- **不可 checkpoint 的操作**: 含有副作用 (如 dropout 的随机状态) 的层需要特殊处理, 否则重新计算时行为会不一致。
- **与 AMP 的交互**: 在混合精度训练中使用 checkpointing 时, 需要确保 autocast 上下文在 checkpoint 段内正确传播。

Activation Checkpointing 的内存收益实例

以 70B 模型 (约 80 层 Transformer) 为例, 假设每层激活值约占 200 MB (取决于 batch size 和序列长度):

- 不做 checkpointing: $80 \times 200 = 16,000 \text{ MB} = 16 \text{ GB}$
- 每 $\sqrt{80} \approx 9$ 层 checkpoint: $9 \times 200 = 1,800 \text{ MB} \approx 1.8 \text{ GB}$
- 代价: 约 33% 额外前向计算

这意味着可以节省约 14 GB 激活值内存, 代价是每步训练时间增加约 33%。在 GPU 内存紧张时, 这个权衡几乎总是值得的。

5.18 混合精度 + Activation Checkpointing 的协同

在实际大规模训练中, 混合精度和 activation checkpointing 通常同时使用, 形成一套完整的内存优化方案:

优化技术	内存节省	计算开销	实现难度
混合精度 (bf16)	激活减半	无（反而加速）	低
Activation checkpointing	激活减 $\sqrt{L} \times$	+33% 前向	中
梯度累积	等效大 batch	无额外开销	低
模型并行	线性切分	通信开销	高

表 30: 常见内存优化技术的对比

6 总结与延伸

6.1 本讲知识体系

本讲把”训练一个模型”拆成了可操作的最小单元：

- 张量如何占内存——从 dtype、numel 到 GPU 内存层次；
- 矩阵乘法如何计算 FLOPs——从 $2BDK$ 到 $6ND$ 规则；
- 参数如何封装进 `nn.Module`——从 `nn.Parameter` 到 `state_dict`；
- 数据如何切 batch——`memmap`、pinned memory、异步拷贝；
- 优化器、训练循环、checkpoint 和混合精度如何协同工作。

6.2 核心公式速查

场景	公式	用途
张量内存	$\text{numel} \times \text{element_size}$	单个 tensor 内存估算
矩阵乘法 FLOPs	$2BDK$	单次 matmul 计算量
训练总 FLOPs	$6ND$	完整训练计算预算
训练时间	$C / (\text{GPUs} \times \eta \times \text{Peak})$	Wall-clock 估算
AdamW 静态内存	$16N$ bytes	混合精度下每参数
Transformer 每层参数	$\approx 12d^2$	模型规模速算
Activation checkpointing	$\sqrt{L} \cdot a$ 内存, +33% 计算	内存优化评估

表 31: 本讲核心公式速查表

6.3 从本讲到后续课程的衔接

本讲内容	后续延伸	对应课程
内存核算	模型并行、张量并行、流水线并行	Lecture 5 (GPUs)
FLOPs 估算	Scaling laws、compute-optimal 训练	Lecture 4 (Scaling)
<code>nn.Module</code>	完整 Transformer 实现	Assignment 1
混合精度	分布式混合精度、FSDP	Lecture 5 (GPUs)
Checkpoint	分布式 checkpoint、容错训练	高级话题

表 32: 本讲内容在课程体系中的位置

最后要记住的一句话

深度学习系统不是“把模型写出来”就结束了，而是要持续回答三个问题：

1. 这个东西占多少内存？
2. 这个东西消耗多少 FLOPs？
3. 这个东西能不能稳定地、可复现地训练起来？

做完 Assignment 1 之后，这些概念会变得更具体，也会更牢固，因为你会亲手把它们用在真正的 Transformer 上。

6.4 拓展阅读

- **混合精度训练原始论文**：Micikevicius et al., “Mixed Precision Training” (2018). 提出了 loss scaling 和 master weights 的混合精度框架。
- **AdamW 论文**：Loshchilov & Hutter, “Decoupled Weight Decay Regularization” (2019). 阐明了解耦权重衰减与 L2 正则化的区别。
- **Activation Checkpointing 原始论文**：Chen et al., “Training Deep Nets with Sublinear Memory Cost” (2016). 提出了 $\sqrt{L}$ 策略的理论分析。
- **FP8 训练**：Micikevicius et al., “FP8 Formats for Deep Learning” (2022). NVIDIA 提出的 E4M3/E5M2 FP8 标准。
- **PyTorch 内部机制**：Edward Z. Yang 的博客 “PyTorch internals”. 详细讲解了 tensor storage、autograd 和 dispatch 机制。
- **Scaling Laws**：Kaplan et al., “Scaling Laws for Neural Language Models” (2020). 建立了参数量、数据量、计算量与 loss 之间的幂律关系。
- **Chinchilla**：Hoffmann et al., “Training Compute-Optimal Large Language Models” (2022). 给出了更精确的 compute-optimal 训练配方。